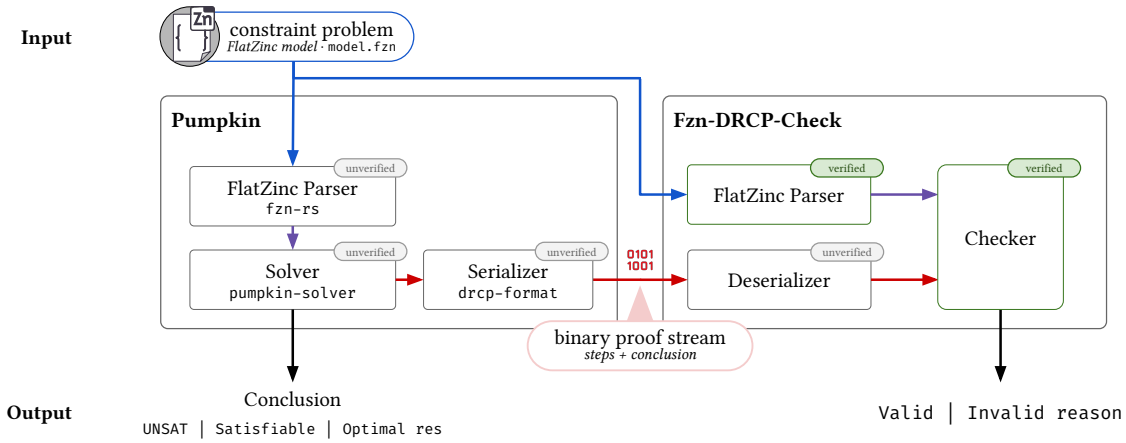
Streaming Verification of Constraint-Optimization Proofs

Julian Marmier

Master's Semester Project
SYSTEMF
Spring 2026

Supervised by
Clément Pit-Claudel
Alexandre Pinazza



Abstract

Pumpkin is a constraint programming (CP) solver for constraint satisfiability and optimization problems that produces proofs of how the solution was derived [1]. DRCP, Pumpkin's proof format, is built on a formal proof system which can be adopted by other CP solvers. FznDrcpCheck is one such checker for DRCP proofs that is implemented and verified in Rocq. The current execution flow is very sequential: the user must first wait for the solver to finish running on their model, and once finished must wait again for the verifier to validate the proof. We speed up the end-to-end execution flow by introducing a binary format for DRCP and adapting FznDrcpCheck to read proofs incrementally from a stream. This yields an average 59% speedup and significant memory usage improvements with small examples compared to the original pipeline.

Table of Contents

1. Introduction 2
2. Background 3
   2.1. DRCP Format 3
   2.2. Proof Stages & Scaffolding 3
3. Design 4
   3.1. Binary Format for DRCP 4
   3.2. Architecture Overview 4
4. Implementation 4
   4.1. Binary serialization format 4
   4.2. Pumpkin 5
   4.3. FznDrcpCheck 5
5. Evaluation 6
6. Discussion 8
   6.1. Results 8
   6.2. Choosing the Serialization Framework .8
   6.3. Contributions to Public Repositories . . 9
7. Related Work 9
8. Timeline 9
9. Conclusion 9
References 10

1. Introduction

Constraint programming (CP) is a paradigm used to solve large and complex combinatorial problems, such as scheduling and resource allocation.

A well-known problem in constraint programming however is the uncertainty in the soundness of the result produced by the solver. Take the following constraint problem presented in [2] :

$$\begin{aligned} \text{assign } x \in \{0,1,2\},\ y, z \\ \text{s.t. } c_1 :\ 5x + 3y \leqslant 6 \\ c_2 :\ x + y + z \geqslant 3 \end{aligned}$$

Both a human and a constraint solver will claim that this problem is unsatisfiable, and it would be reasonable to ask of the human to show their work. But what about the constraint solver, which could contain bugs? With proof checking, a solver returns a proof containing the steps it made to get to its conclusion, which can be verified by a checker. The checker then either accepts or rejects the proof, the latter case indicative of unsound reasoning steps on behalf of the solver and attesting to the likely presence of software bugs.

However, providing greater guarantees on the soundness of the result generally entails a significant sacrifice in memory and execution time. During the search for a solution, CP solvers may learn impossible variable assignments and constraints, known as nogoods. Proof frameworks in CP solvers make use of the negation of nogoods—referred to as *deductions*—to derive a chain of reasoning that can be used to justify the steps the solver made to get to the final solution, using a method known as Reverse Unit Propagation [3]. Many of these nogoods are redundant and result in proofs on the order of hundreds of gigabytes, as well as significant overhead caused by the logging operation. On top of that, proof checkers are linear in the size of inputs, which results in higher checking runtime as well.

Pumpkin [1] is a constraint-solving framework developed by the Constraint Solving Lab at TU Delft which addresses performance and memory issues presented by previous proof logging approaches, documented in Section 2.2.

Deletion Reverse Constraint Propagation (DRCP) is a proof system that formalizes Pumpkin's proof logging by introducing a solver-agnostic format that can be adopted by other projects. The FlatZinc–DRCP Checker (FznDrcpCheck) [2] reads a FlatZinc model and a DRCP proof and checks the proof for soundness. FznDrcpCheck is implemented and formally verified with Rocq by virtue of DRCP's formal proof system, providing a proof of concept for trusted and practical proof logging for CP.

Data serialization converts language-specific data structures into a compact binary format that can be easily uncompressed back to the original structure in another language, allowing for memory- and latency-efficient transfer and storage of data. Data serialization frameworks such as Cap'n Proto [4] and Protocol Buffers (colloquially known as Protobuf) [5] in particular are well known for their speed and memory efficiency.

Although Pumpkin and the DRCP format and checker already make significant advances in the fields of proof logging for CP, the question of further improving proof-checking speed still remains. This project makes two contributions:

1. a binary version of the DRCP format based on Protocol Buffers, with a Rust implementation as part of the Pumpkin suite that enables reading and writing in the format;
2. a memory-efficient and verified implementation of FznDrcpCheck adapted for real time streaming of proof data, both in binary and text.

			`a 1 [y >= 0]`
			`a 2 [x <= 1]`
c_3	$c_1 \vDash \langle y \geq 0 \rangle \rightarrow \langle x \leq 1 \rangle$	$\left\{ \begin{matrix} \text{Linear} \\ \text{Bounds} \end{matrix} \right\}$	`i 3 1 0 2 l:linear_bounds`
			`a 4 [x >= 1]`
			`a 5 [y <= 0]`
c_4	$c_1 \vDash \langle x \geq 1 \rangle \rightarrow \langle y \leq 0 \rangle$	$\left\{ \begin{matrix} \text{Linear} \\ \text{Bounds} \end{matrix} \right\}$	`i 6 4 0 5 l:linear_bounds`
			`a 7 [z <= 1]`
			`a 8 [y <= 1]`
c_5	$c_2 \vDash \langle x \geq 1 \rangle \rightarrow \langle y \geq 1 \rangle$	$\left\{ \begin{matrix} \text{Linear} \\ \text{Bounds} \end{matrix} \right\}$	`i 9 4 7 0 8 l:linear_bounds`
c_6	$c_3, c_4, c_5 \vDash \langle x \geq 1 \rangle \wedge \langle y \geq 1 \rangle \wedge \langle z \leq 1 \rangle \rightarrow \bot$	$\{ \text{Deduction} \}$	`n 10 4 1 7 0 3 6 9`
			`a 11 [X <= 0]`
c_7	$c_6 \vDash \langle y \geq 0 \rangle \wedge \langle z \leq 1 \rangle \rightarrow \langle x \leq 0 \rangle$	$\{ \text{Nogood} \}$	`i 12 10 0 1 7 c:10 l:nogood`
			`a 13 [X >= 1]`
c_8	$c_2 \vDash \langle y \leq 1 \rangle \wedge \langle z \leq 1 \rangle \rightarrow \langle x \geq 1 \rangle$	$\left\{ \begin{matrix} \text{Linear} \\ \text{Bounds} \end{matrix} \right\}$	`i 14 13 0 8 7 c:6 l:linear_bounds`
c_9	$c_7, c_8 \vDash \langle y \geq 0 \rangle \wedge \langle y \leq 1 \rangle \rightarrow \langle z \leq 1 \rangle \rightarrow \bot$	$\{ \text{Deduction} \}$	`n 15 1 8 7 0 10 12`
c_{10}	$\top \vDash \top \rightarrow \langle y \geq 0 \rangle$	$\left\{ \begin{matrix} \text{Problem} \\ \text{domains} \end{matrix} \right\}$	`i 16 0 1 l:initial_domain`
c_{11}	$\top \vDash \top \rightarrow \langle y \leq 1 \rangle$	$\left\{ \begin{matrix} \text{Problem} \\ \text{domains} \end{matrix} \right\}$	`i 17 0 8 l:initial_domain`
c_{12}	$\top \vDash \top \rightarrow \langle z \leq 1 \rangle$	$\left\{ \begin{matrix} \text{Problem} \\ \text{domains} \end{matrix} \right\}$	`i 18 0 2 l:initial_domain`
c_{13}	$c_1, c_8, c_7 \vDash \langle x \geq 1 \rangle \wedge \langle y \leq 1 \rangle \wedge \langle z \leq 1 \rangle \rightarrow \bot$	$\{ \text{Nogood} \}$	`i 19 1 8 7 c:15 l:nogood`
			`c UNSAT`

Table 1. Proof of unsatisfiability (*left*) next to its translation in DRCP (*right*).

2. Background

2.1. DRCP Format

Recall the example constraint problem from Section 1. Table 1 presents what a proof of unsatisfiability would look like alongside its translation in DRCP. Each line gets a separate *step ID* (the second space-separated value) unique throughout the proof. Constraints of the form *variable <comparator> constant*, known as *atomic constraints*, are introduced with the letter `a`. These are then reused by inference and deduction steps. The former, denoted by `i`, encodes the propagation of new constraints from former ones, accompanied by the inference rule that enables this. The latter, denoted by `n`, encodes nogoods as the result of a deduction chain of previous inferences. Finally the conclusion is denoted by `c` and noted as either a dual bounded conclusion based on another constraint or `UNSAT` for unsatisfiable problems.

2.2. Proof Stages & Scaffolding

As shown by the horizontal lines in Table 1, the solver emits proof steps in stages of multiple inferences leading to a deduction. Pumpkin can be configured to emit either full proofs or incomplete proofs known as scaffolds which contain only derived nogoods.

These inferences can be re-derived afterwards by a proof checker, which allows the processor to discard unused proof stages on the fly. This is particularly useful in the case of large constraint models which can cause the solver to branch into cases irrelevant to the conclusion, avoiding expensive inference logging operations. While this approach may reduce solving time and proof disk usage, inference expansion is also a time-costly operation that slows the end-to-end solving & checking process.

3. Design

3.1. Binary Format for DRCP

The format presented in Section 2.1 excels at providing a format both legible for humans while remaining relatively compact in terms of size, however it causes performance bottlenecks in terms of serializing and parsing, which is rather expensive. The binary format which we detail in Section 4.1 takes the same fields as DRCP and compresses them into a format that enables much faster reading and writing as the format better matches the internal data representations of Pumpkin and FznDrcpCheck.

3.2. Architecture Overview

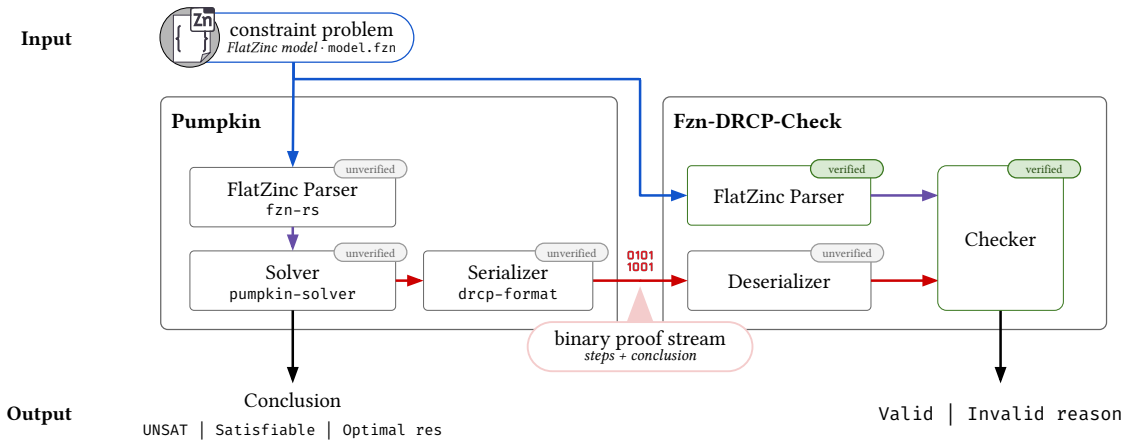


Figure 1. Proposed architecture for binary streaming between Pumpkin and FznDrcpCheck

Figure 1 presents an overview of the architectures of Pumpkin and FznDrcpCheck, as well as how they work together when streaming proofs. The user begins by introducing a constraint problem encoded as a FlatZinc model [6] and a shared proof location to both programs. Pumpkin and FznDrcpCheck parse the model, after which FznDrcpCheck waits until it receives serialized data from the solver, and passes it incrementally to the Rocq as a lazy list. Pumpkin logs data while it is solving the constraints, which allows for the proof to be checked on the fly and aborted if an unsound inference is encountered ahead of time. Once the solver finds its conclusion, it stores it in the stream and displays it to the user along with a message indicating that the proof is valid.

4. Implementation

4.1. Binary serialization format

We chose Protocol Buffers [5] as the binary serialization framework for the DRCP format. We opted for a well-known serialization framework rather than an ad-hoc binary format for a few reasons. First, it avoids the complexity of implementing a parser that might not be efficient or as well thought out as

DRCP Specification

```
n <step_id> <atomic constraint ids> [0 <propagation hint>]
```

Protocol Buffers Schema

```
message NogoodStep {
  uint32          id              = 1;
  repeated sint32 literals        = 2;
  repeated uint32 propagation_hint = 3;
}
```

Listing 1. DRCP Specification for the encoding of a *Nogood* step compared to its definition in our Protocol Buffers schema.

```
Inductive LazyProofTrace :=
| SConcl (e: ProofFact)
| SCons (hd: ProofStage)
        (tl: LazyProofTrace)
| SThunk (next: unit -> LazyProofTrace).
```

```
type ('a, 'b) lazyProofTrace =
  | SConclusion of 'b
  | SCons of 'a * ('a, 'b) lazyProofTrace
  | SThunk of
      (unit -> ('a, 'b) lazyProofTrace)
```

Listing 2. Lazy proof trace inductive defined in Rocq (*left*) and its corresponding OCaml extraction (*right*). The OCaml version is kept generic to avoid conflicts with Rocq namespaces, but it is ultimately instantiated as a `(ProofStage, ProofFact) lazyProofTrace`.

those already made available in well-established projects, compromising speed and reliability. Second, we wanted to ensure that changes made to the DRCP format could easily be ported to the binary side. Listing 1 shows part of the DRCP Format specification next to its respective Protocol Buffers specification. The latter spec improves developer experience by providing a single source of truth for the binary format both for Pumpkin and for FznDrcpCheck, allowing for better developer experience through strict type checking in the derived (de)serializers in Rust and OCaml respectively.

4.2. Pumpkin

The Pumpkin project maintained by the ConSol lab [1] is meticulously maintained and contains a codebase organized into separate interoperable Rust crates. We implemented the majority of our changes in the `drcp-format` crate by generalizing the `ProofReader` and `ProofWriter` structs to respective `ProofRead` and `ProofWrite` Rust traits and implementing `BinaryProofReader`/`BinaryProofWriter` structs alongside the original text-based structs. Minimal changes were made to the `pumpkin-proof-processor` and `pumpkin-solver` crates to add command-line parameters for reading and writing proofs in the binary format.

4.3. FznDrcpCheck

Also maintained by the ConSol lab, FznDrcpCheck's codebase required larger changes to its codebase in order to support reading in the binary format as well as streaming of data.

Listing 2 shows the `LazyProofTrace` inductive as defined in Rocq alongside its respective `lazyProofTrace` extraction in OCaml, which consists in an ad-hoc lazy list. Lazy lists form the base of the checker's streaming capabilities. By virtue of the guarantee that a proof will always eventually finish with a conclusion, we represent the proof as a lazy list of stages. Laziness is introduced by calling a thunk pointing to code in OCaml, as described in the next paragraph. To some extent, this introduces unsoundness in the checker as termination is not guaranteed in OCaml, but we argue that this termination is dependent on the termination of the solver, and thus stricter termination guarantees would have little impact in the case of streaming.

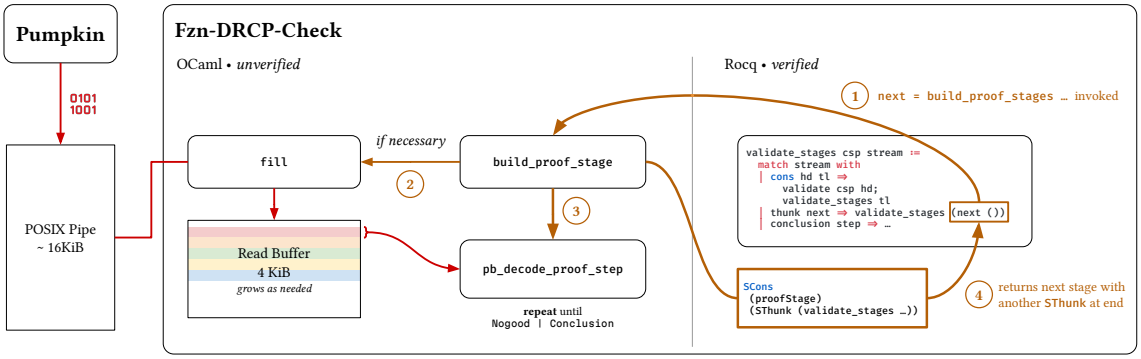


Figure 2. Streaming of binary data between Pumpkin and FznDrcpCheck

Figure 2 demonstrates how proof steps are passed from Pumpkin to FznDrcpCheck and processed accordingly. As Pumpkin begins to solve a given constraint problem, it writes proof data to a POSIX pipe. In parallel, the checker begins checking a `LazyProofTrace` that contains an `lthunk next` pointing to `build_proof_stage` (1). Each call to `build_proof_stage` fills the read buffer from the pipe and resizes it as needed (2), and decodes steps until a deduction is reached. It pre-processes said stage and returns `Cons stage (Thunk next)` where `next` points back to `build_proof_stage`. The checker validates the stage and once done calls `next` again, repeating this process until `build_proof_stage` returns a `Single stage` containing the conclusion, which the checker verifies to certify the proof as valid. If any proof stage is unsound, the checker immediately interrupts execution and prints an error, allowing the user to detect bugs in the solver before having to wait for the entire proof to be processed.

```
Module Type CheckerSig.
  Parameter Proof : Type.
  Parameter CheckErrType: Type.
  Parameter InferenceRule : Type.

  Parameter validate: (string -> option InferenceRule) ->
    ConstraintProblem -> Proof -> CheckResult CheckErrType.
  Parameter soundness : forall rules,
    checker_sound Proof CheckErrType (validate rules).
End CheckerSig.
```

Listing 3. Checker contract as defined in FznDrcpCheck's Rocq specification.

Supporting streaming required small changes to the Rocq specification of the checker. Listing 3 shows the generic signature each checker must implement. We modularized the original implementation into a `CheckerImpl` section containing the shared implementation and a `SequentialChecker` module. The latter contains functions dependent on the `Proof` parameter, which in this case is `list proofStage`. We added a `StreamedChecker` with a lazy list of proof stages as its `Proof` type, reusing the necessary functions from `CheckerImpl` and keeping code duplication to a minimum.

When logging the full proof directly and bypassing the processing phase, certain stages can be very large. One particular case contains up to 5 million inference steps, leading up to a deduction step containing a list of 5 million references to those inferences. This caused two problems with the checker:

1. Non tail-recursive functions such as `List.map` caused stack overflows when called on such lists. We mitigated the overflows by replacing calls to `List.map` with `List.rev_map` followed by `List.rev`. Both are tail-recursive and remain linear amid the chaining being slower than the original `List.map`.
2. The `hydrate_deduction_chain` function had to recurse over the entire deduction chain and replace references to inference steps from the deduction step with their actual values. Not only did the original implementation lack tail-recursion, but it was also quadratic due to its use of the linear-time `List.filter` on the list of inferences, making the function inefficient for large lists. Clément solved this by replacing the list of inferences with a modular finite map [7], thus decreasing search time to $\log(i)$, i being the size of the list of inferences.

5. Evaluation

We evaluate the performance of our pipeline using constraint satisfaction and optimization problems from the MiniZinc challenge [8]. Because proofs are not saved to disk when streaming, we focus on the streaming between `pumpkin-solver` and FznDrcpCheck. To that end, we run three different pipeline configurations, each targeting both the text (T) and binary (B) formats:

1T/B Pumpkin solver emits full proof, then proof is checked by FznDrcpCheck (baseline)

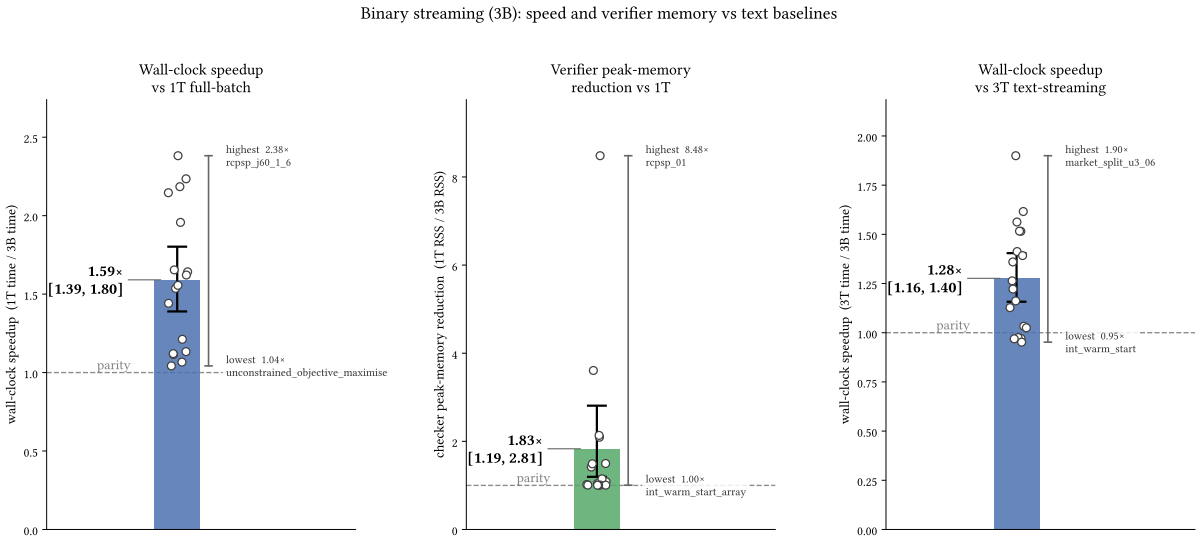


Figure 3. Time and memory efficiency gains comparing the original text pipeline with the streamed binary pipeline (left/center), and speedup comparison between text and binary streaming (right).

2T/B Pumpkin solver emits proof scaffold, is processed by `pumpkin-solver` and passed to FznDrcpCheck.

3T/B Pumpkin solver emits full proof to a pipe while it is checked by FznDrcpCheck (streaming).

All tests were run on small proof examples on a 2020 M1 MacBook Pro (16GB memory).

Figure 3 shows our results for 17 small examples. On average, the new binary pipeline is 1.59 times faster, reaching sometimes double the speed of the original implementation, and never below parity. Peak memory is also significantly lower, with a 1.83 times reduction in usage on average, going up to 8.48 times for one case in particular. Comparing just binary to text both applied to streaming, we see that the binary format provides a 1.28 times speedup compared to text.

A larger example covering all six test cases is shown in Figure 4. The proof processing pipeline (2) is significantly slower than all others in both the text and binary cases, and consumes more than twice the memory of even the full proof. The streaming pipeline (3) is faster than its batch counterpart in

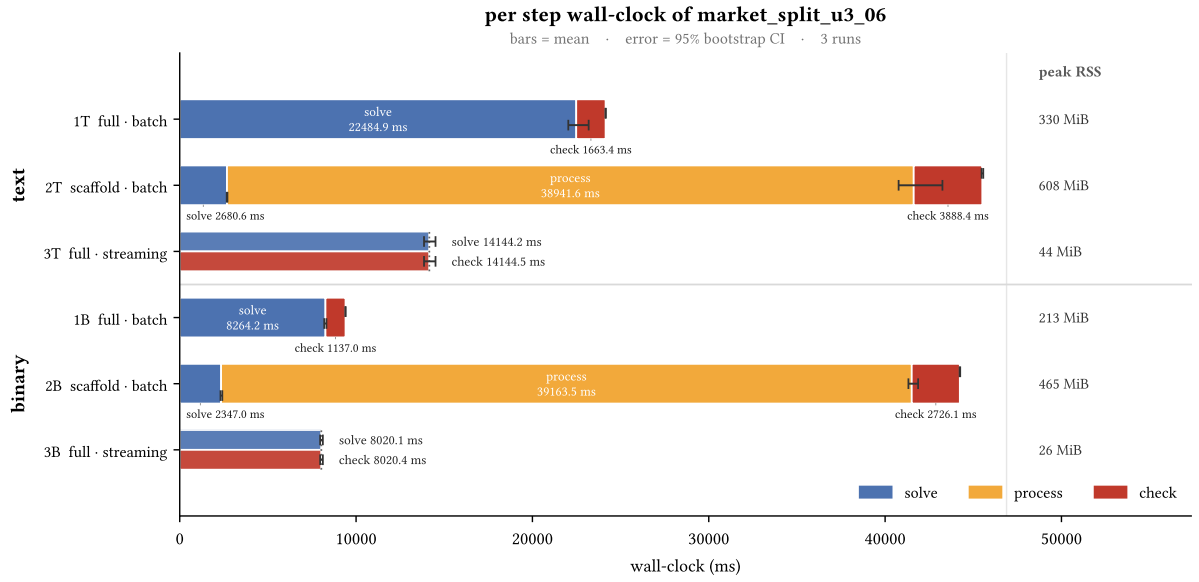


Figure 4. End-to-end run comparison on a medium-sized constraint satisfaction problem.

both cases, and uses around 6-10 times less memory. Overall, the binary format is faster than the text format, albeit yielding minimal gains on the scaffold pipeline.

6. Discussion

6.1. Results

The results suggest that our implementation significantly improves the usability of the checker, not just by speeding up the proof checking process, but also by using less memory. With the original implementation, large unprocessed proofs would cause the checker to run out of memory. With our implementation, the same problem can be solved and checked using just a fraction of the resources.

6.2. Choosing the Serialization Framework

We considered using the Cap'n Proto framework [4] to take advantage of its no-copy deserialization, which simply accesses data by pointing to its location in memory rather than copying into an intermediate buffer and decoding it (which is done in Protocol Buffers). However, as explained in section Section 2.2, the checker first needs to preprocess proofs by injecting atomic constraints into inference and deduction steps, requiring that the additional data be copied and thus canceling out the memory-saving benefits of Cap'n Proto unless large changes were made to the binary format at the expense of direct compatibility with DRCP. Figure 5 compares the size and checking time of the same proof between DRCP text, Cap'n Proto using compression and without compression, and Protocol Buffers. Cap'n Proto is slower in both cases when copying data and has a larger memory footprint. Protocol Buffers uses a special form of integer encoding known as `Varint` which packs integers into only as many bytes as needed to represent them: for example the `ID` field in a step might have a value of 125, and would be encoded on a single byte instead of the usual 4 bytes needed for a 32-bit integer. This motivated the choice of using Protocol Buffers.

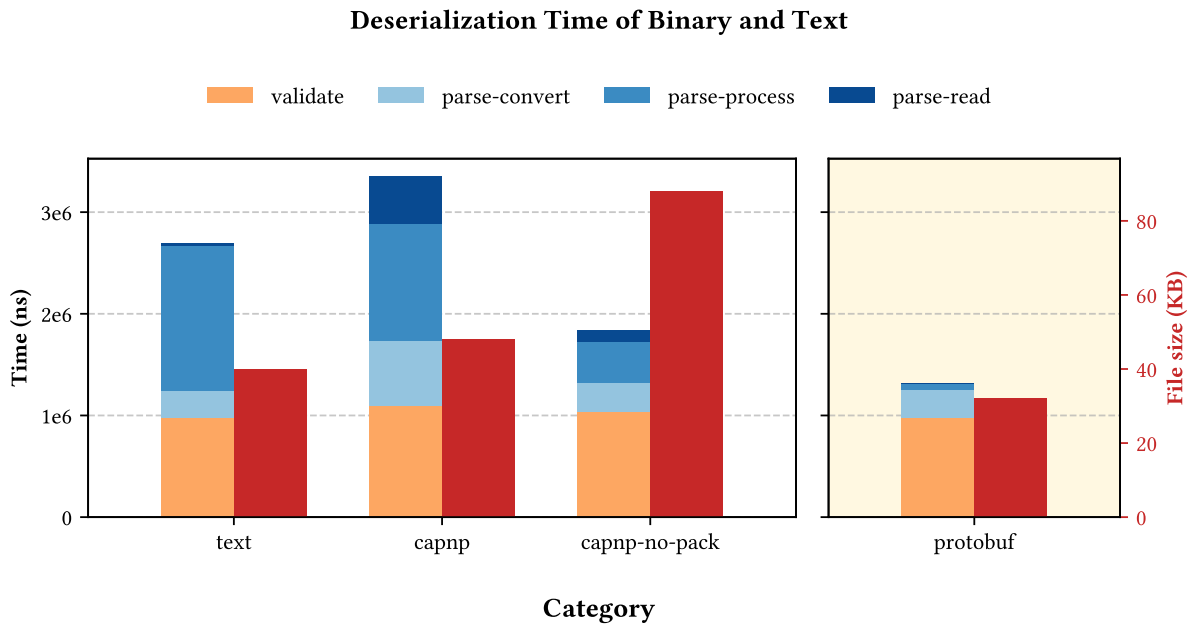


Figure 5. Comparison between time spent (left axis) and proof format footprint (right axis)

6.3. Contributions to Public Repositories

We are planning to introduce the implementations described in Section 4 as pull requests to the repositories of Pumpkin and Fzn-DRCP-Check. As of the time of writing, a few adjustments still need to be made before submitting them.

7. Related Work

While the concept of proof logging (and consequently proof streaming) is relatively new to CP, it has been around for longer in the context of SAT solvers where proofs of unsatisfiability are also mandated for reasons similar to those in CP. Research investigating the scalability of unsatisfiability proofs in a distributed setting [9] shows that it is possible to merge partial proofs, however "proof-streaming techniques to eliminate most I/O" are only mentioned as a potential improvement in reducing overhead, without further development on the topic.

8. Timeline

Milestone	Description	Planned Weeks	Actual Weeks
0	Project proposal & presentation	1	1
1	DRCP binary schema	2–3	2–10
2	Standalone DRCP ↔ binary converter	4–6	4–6
3.1	Data streaming into Pumpkin	7–8	10–11, 17
3.2	Data streaming into FznDRCPCheck	9–12	7–17
4	Evaluation, discussion & refactoring	14	15-18

Table 2. Project timeline organized by milestone, comparing the planned timeline to the actual timeline.

Overall the project achieved its initial goals, with certain parts slower than planned. Table 2 shows the planned milestones of the project and compares their projected completion with their actual completion. Finding the right binary schema was a grand feat of trial and error, taking significantly more time than expected. We had initially gone in the Cap'n Proto direction up until about Week 10, but after extensive testing we ended up going with Protocol Buffers instead. Milestone 3.1 started behind schedule, but took the same amount of time. On the other hand, Milestone 3.2 was a rather involved task and took a significantly larger amount of effort than expected, accounting for more than half of the project's timeline. The project ran about 4 weeks behind schedule and took around 18 weeks to complete instead of the initially planned 14.

9. Conclusion

With this project, we build upon the Pumpkin project's original inspiration for efficient proof logging in constraint programming in two steps. First, we increase usability of the proof format by extending it to a faster yet isomorphic binary format. Second, we improve the end-to-end speed of the solving and checking process, the latter being formally verified. We hope this will help contribute to making proof logging standard for CP solvers.

References

[1] M. Flippo, K. Sidorov, I. Marijnissen, J. Smits, and E. Demirović, "A Multi-Stage Proof Logging Framework to Certify the Correctness of CP Solvers," in *30th International Conference on Principles and Practice of Constraint Programming (CP 2024)*, P. Shaw, Ed., in Leibniz International Proceedings in Informatics (LIPIcs), vol. 307. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024, p. 11:1–11:20. doi: 10.4230/LIPIcs.CP.2024.11.

[2] Konstantin Sidorov, Maarten Flippo, Tip Ten Brink, Clément Pit-Claudel, and Emir Demirović, "Certified Verification of Constraint Programming Proofs," *Unpublished*.

[3] A. Gelder, "Verifying RUP Proofs of Propositional Unsatisfiability.," Jan. 2008.

[4] K. Varda, "Cap'n Proto: A Fast Data Interchange Format and Capability-Based RPC System." [Online]. Available: https://capnproto.org/

[5] Google Inc., "Protocol Buffers: Google's Data Interchange Format." [Online]. Available: https://developers.google.com/protocol-buffers

[6] N. Nethercote, P. J. Stuckey, R. Becket, S. Brand, G. J. Duck, and G. Tack, "Minizinc: Towards a Standard CP Modelling Language," in *International Conference on Principles and Practice of Constraint Programming*, Springer, 2007, pp. 529–543.

[7] P. Letouzey and A. W. Appel, "MMaps: Modular Finite Maps over Ordered Types." [Online]. Available: https://github.com/rocq-community/mmaps

[8] MiniZinc, "Mzn-Challenge: An Archive Repository Containing the MiniZinc Instances Used for the MiniZinc Challenge." [Online]. Available: https://github.com/MiniZinc/mzn-challenge

[9] D. Michaelson, D. Schreiber, M. J. H. Heule, B. Kiesl-Reiter, and M. W. Whalen, "Producing Proofs of Unsatisfiability with Distributed Clause-Sharing SAT Solvers.," *Journal of automated reasoning*, vol. 69, no. 2, p. 12, 2025, doi: 10.1007/s10817-025-09725-w.